Day 10

Embeddings and vector databases

Day 09 gave us clean **chunks**. Today we make them **searchable by meaning**, then store them in **FAISS**, **Qdrant** and **Pinecone**. On the 198 BNS chunks of **Kanoon.ai**.

1 What is an embedding
2 Why embed
3 How to embed
4 How an index works
5 Vector databases
6 FAISS
7 Qdrant
8 Pinecone
9 Which one when

1 · what is an embedding

GPS coordinates, but for meaning

Places

Two numbers each. **Close places have close numbers.**

```
Delhi [28.61, 77.21]
Gurugram [28.46, 77.03]
Chennai [13.08, 80.27]

Delhi → Gurugram: 0.23
Delhi → Chennai : 15.83
```

Sentences

Imagine 3 scores: food, money, law.

sentence	food	money	law
I ordered a pizza	0.9	0.1	0.0
food delivered home	0.8	0.2	0.0
stock market fell	0.0	0.9	0.1
murder is punished...	0.1	0.2	0.9

embedding: a list of numbers that represents the **meaning** of text. A real model uses **hundreds or thousands** of numbers and **learns** them. Nobody decides what each one means.

1 · a real one

1,536 small numbers, length 1

```
client.embeddings.create(model="text-embedding-3-small",
                         input=sentences)

one vector : 1536 numbers
first 8    : [-0.017 -0.027 -0.073 -0.035 ...]
all of them: (4, 1536)          # shape = (texts, dims)
smallest -0.096 | biggest 0.12 | length 1.0
```

how close (1 = same meaning)	pizza
food delivered to my home	0.49
the stock market fell	0.23
murder is punished...	0.06

Rules

- **dimensions**: small 1,536 · large 3,072 · MiniLM 384
- **one number alone means nothing**; only whole vectors, compared
- vectors from **different models can't be compared**: same model for chunks and questions
- memory: 4 bytes per number

  `10 M chunks × 1,536 × 4 =` **`61 GB`**

pizza vs food delivery: almost no shared words, closest pair.

2 · why embed

Users don't type the words of the law

Keyword search

```
'killing someone' → []
'murder' → bns::1, ::9, ::10…
'hit someone and…' → []
```

The BNS says **"causes death"**, not "killing". The **vocabulary gap**.

Meaning search

question	top chunk	page
Is killing someone a crime?	bns::58	32
I hit someone, they got hurt	bns::69	38
phone stolen on the metro	bns::151	79

Murder, grievous hurt, theft. **No shared words needed.**

```
scores = chunk_vectors @ embed([query])[0]     # embed every chunk once, the question each time
best = np.argsort(-scores)[:3]                  # nearest = most similar meaning
```

2 · where embeddings fail

Similar meaning, not exact precision

embeddings are good at	embeddings are bad at
synonyms, paraphrases ("killing" ≈ "causes death")	exact strings: `Section 103`, `INV-2023-091`, error codes
questions in the user's own words	words the model rarely saw: Hinglish, rare legal terms
typos, word order	"not", numbers, dates: "Type 1" vs "Type 2" look the same

```
search("Section 103") → bns::5, bns::6, bns::3 (pages 3-4)
search("hatya ki saza kya hai") → bns::159, bns::73, bns::151 (wrong)
Section 103 lives in bns::61, page 33. Neither finds it.
```

Day 11: keyword search (BM25) + vectors = **hybrid search**. Embeddings also power recommendations, clustering, duplicate detection, classification.

3 · measuring close

Cosine, dot product, distance

measure	what	pizza vs food
cosine	angle between vectors	`0.4934`
dot product	`sum(a * b)`	`0.4937`
euclidean	straight-line distance (0 = same)	`1.007`

OpenAI vectors have **length 1**, so cosine = dot product and all three give **the same ranking**.

In practice

- use what the **model card** says, everywhere
- length-1 vectors: dot product is the cheapest
- cut vectors yourself (`v[:256]`)? **normalise again**
- the DB needs to know it: `Distance.COSINE`, `metric="cosine"`, `IndexFlatIP`

3 · which model

Test on your own questions

model	where	dims	max input	price
text-embedding-3-small	OpenAI API	1,536	8,192 tokens	$0.02 / 1M
text-embedding-3-large	OpenAI API	3,072	8,192 tokens	$0.13 / 1M
all-MiniLM-L6-v2	your laptop	384	256 word pieces	free

question → pages of top 3	right	small	large	MiniLM
Is killing someone a crime?	31–32	32 31 35	32 31 33	17 33 31
hit someone, they get hurt	38–39	38 39 38	38 39 40	44 39 40
phone stolen on the metro	78–79	79 85 78	79 78 78	80 22 78
five people, one murder	33	7 7 33	7 35 33	82 17 34

MiniLM reads only the first 256 word pieces of our ~500-token chunks. Leaderboard: MTEB. **Don't pick from a leaderboard alone** (Day 13 measures it).

3 · shorter vectors · cost

Smaller vectors, tiny bills

Matryoshka: `dimensions=256`

The first numbers carry the most meaning, like Russian dolls.

```
top 5, 1,536 dims: 58 57 64 60 10
top 5, 256 dims: 57 58 64 62 60
```

4 of 5 the same, **6× less memory**.

Cost: the whole BNS

```
101,637 tokens
small: $0.0020
large: $0.0132
```

Less than a rupee. You pay **once at ingestion**, then one short embedding per question.

rules: **batch** (100 texts per call) · embed **breadcrumb + text** · store the **model name** (new model = re-embed all) · **cache**, never embed twice

4 · how an index works

Don't compare with every vector

Flat: compare with all

```
10,000 vectors: 3.9 ms
100,000 vectors: 43.2 ms
500,000 vectors: 696.3 ms
```

Exact, but grows with your data. 100 M chunks × 1,000 users: too slow.

IVF: shelves in a library

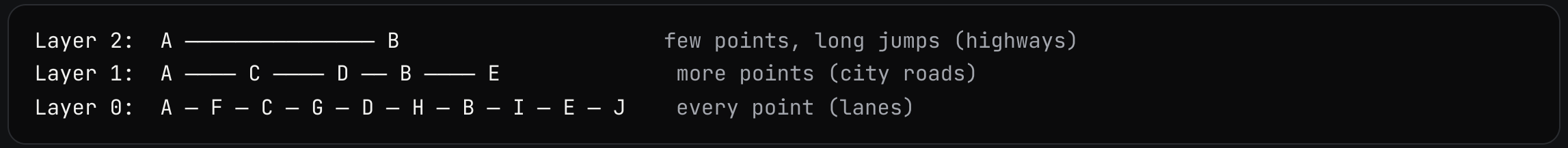
- **build once**: k-means groups vectors into clusters
- **search**: nearest few centres, then only their vectors
- ours: 100,000 vectors, 300 clusters, `n_probe=3`

```
compared 1.0%, found 10/10 of the true top 10
```

ANN: approximate nearest neighbour. Look at a small part, and accept that **very rarely** the true best is missed.

4 · HNSW · quantization

A graph of neighbours, like roads



Start at the top, jump to the neighbour closest to the question, go down, repeat. Knobs: `m` (links per point), `ef_construct` (build effort), `ef` (search effort). Qdrant default: `m=16, ef_construct=100`.

index	how	good	bad
Flat	compare with all	exact, no build	slow at scale
IVF	search a few clusters	fast, less memory	needs training, misses at edges
HNSW	walk a neighbour graph	very fast, very accurate	more memory, slow build
+ Quantization	1 byte or 1 bit per number	4–32× smaller	a little less accurate

5 · vector databases

A numpy file is fine for 198 chunks

need	numpy file	vector DB
fast search on millions	flat, slow	HNSW index
text + metadata with vector	separate JSON	payload
"only BNS", "only my files"	write it yourself	filters
change one chunk	rewrite the file	upsert / delete by id
many users, servers, backups	no	yes

Words

- **point** (Pinecone: record) = `id` + `vector` + `payload`
- **collection** (Pinecone: index) = a SQL table with a vector column
- **upsert** = insert, or overwrite if the id exists
- **stable id** (`bns::58`): re-running ingestion updates, never duplicates

Options: FAISS (library) · pgvector (Postgres) · Chroma (prototypes) · Qdrant (open source) · Pinecone (managed cloud) · Weaviate, Milvus (large scale)

6 · FAISS

A library, not a database

```
import faiss                        # pip install faiss-cpu

index = faiss.IndexFlatIP(1536) # flat, inner product
index.add(vectors.astype("float32"))
scores, ids = index.search(q, 3)
# ids = positions 58, 57, 64 → look up chunks yourself

index.add(vectors)                  # run ingestion again
index.ntotal                        # 396: duplicates

sel = faiss.IDSelectorBatch(property_ids)   # filter: YOU find ids
index.search(q, 3, params=faiss.SearchParameters(sel=sel))

faiss.write_index(index, "cache/bns.faiss")
faiss.IndexHNSWFlat(1536, 32, faiss.METRIC_INNER_PRODUCT)
```

What we saw

- Meta, MIT license, C++ inside your Python
- same chunks and scores as numpy and Qdrant
- flat and HNSW: **same top 5**; change index = one line
- **stores vectors only**: no text, page, id
- no upsert: adding twice = **396**
- Mac + torch: `omp_set_num_threads(1)` or the kernel crashes

7 · Qdrant

Open source, same code from laptop to server

```
qdrant = QdrantClient(path="cache/qdrant")    # or ":memory:", or url=
qdrant.create_collection("bns", vectors_config=
    models.VectorParams(size=1536, distance=models.Distance.COSINE))

point_id = uuid.uuid5(uuid.NAMESPACE_URL, "bns::58")  # same id every run
qdrant.upsert("bns", points=[models.PointStruct(
    id=point_id, vector=v, payload=chunk)])

qdrant.query_points("bns", query=q, limit=3,
    query_filter=models.Filter(must=[models.FieldCondition(
        key="page", range=models.Range(gte=78, lte=91))]),
    score_threshold=0.4)
```

What we saw

- upsert twice: **198 → 198**, no duplicates
- "What is the punishment?": pages 10, 69, 68 → with filter **89, 88, 79**
- `score_threshold=0.4`: "bring my dog to the office?" → **nothing**, not 3 random sections
- server: `docker run -p 6333:6333 qdrant/qdrant`, payload indexes before upload

8 · Pinecone

Fully managed: no server, just an API key

```
pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
pc.create_index("kanoon-bns", dimension=1536, metric="cosine",
    spec=ServerlessSpec(cloud="aws", region="us-east-1"))
index = pc.Index(name="kanoon-bns")

index.upsert(vectors=[{"id": "bns::58", "values": v,
    "metadata": {"page": 32, "act": "bns", "text": ...}}],
    namespace="bns", batch_size=100)

index.query(vector=q, top_k=3, namespace="bns",
    filter={"page": {"$gte": 78, "$lte": 91}},
    include_metadata=True)
```

Notes

- **serverless**: pay for storage and reads/writes
- ids are any string: `chunk_id` as it is
- **namespaces**: one per Act or per customer
- Mongo-style filters: `$eq $in $gte $and`...
- keep metadata small; writes take seconds to be searchable
- free Starter: 2 GB, 5 indexes, AWS us-east-1

9 · which one when

Engine, own car, or Uber

FAISS = a car engine. Powerful, free. You build the seats, doors and fuel tank yourself.

Qdrant = your own car. Complete. Park it at home or rent a spot (cloud). You maintain it.

Pinecone = Uber. Own nothing, book with a key, pay per use. Can't take it home.

	FAISS	Qdrant	Pinecone
what	library	database, open source	database, cloud service
runs	inside your program	laptop, Docker, server, cloud	only Pinecone's cloud
text + metadata	no, vectors only	payload	metadata
ingest twice	duplicates	upsert overwrites	upsert overwrites
filters	ids found in Python	`Filter(must=[…])`	`{"page": {"$gte": 78}}`
cost	free	free self-hosted, cloud free tier	free Starter, then paid
pick it for	research, one script, max speed	real app, data stays with you	real app, nobody runs servers

Kanoon.ai uses Qdrant: a full DB on a laptop with no account, and Indian legal data stays on our server.

wrap up

Recap and homework

Recap

text → vector	`embeddings.create(model, input)`
close or far	dot product (length 1)
shorter	`dimensions=256`
index	flat → IVF / HNSW + quantization
store + search	FAISS · Qdrant · Pinecone
same 4 steps	create · add · search · filter

Homework

- 10 BNS questions in your words: small vs large vs MiniLM
- `dimensions=256` and `512`: when does the top 5 change?
- IVF with `n_probe=1, 3, 10`: plot % compared vs found
- a question where `score_threshold=0.4` drops a right chunk
- FAISS `IndexIVFFlat`: same top 3 as flat?

Sources, checked 2026-10-26/28: developers.openai.com/api/docs/guides/embeddings · developers.openai.com/api/docs/pricing · huggingface.co/sentence-transformers/all-MiniLM-L6-v2 · github.com/facebookresearch/faiss (faiss-cpu 1.15.1) · qdrant.tech/documentation (qdrant-client 1.20.0) · qdrant.tech/pricing · docs.pinecone.io (pinecone 10.2.0) · pinecone.io/pricing · github.com/pgvector/pgvector

All numbers from `day-10/code`, run 2026-10-28. Pinecone code checked against the SDK; run it live with a key. Not legal advice.